Modern Deep Learning Architectures

Goodfellow, Bengio & Courville (2016); Jurafsky & Martin (2024)

Emmanuel Djegou, PhD
emmanueldjegou5@gmail.com

Course Outline

Module 8 — CNNs

1. The convolution operation
2. Sparse interactions
3. Parameter sharing
4. Equivariance
5. Pooling
6. CNN architectures

Module 9 — Sequences

7. Sequential data
8. RNNs
9. BPTT
10. Long-term dependencies
11. LSTMs and GRUs

Module 10 — Transformers

12. Self-attention
13. Multi-head attention
14. Transformer block
15. Language model head
16. Generation & training
17. LLMs and harms

Why Dense Networks Struggle with Images

The scaling problem:

A fully connected layer connects every input unit to every output unit.

- A modest 200×200 RGB image has 120,000 inputs
- A single dense hidden layer of 1000 units needs 1.2×10^8 weights
- This is per layer — the parameter count explodes

The statistical problem:

A dense layer treats each pixel independently. It must *relearn* the same feature (e.g. an edge) separately at every location.

Key structural facts about images:

- Images have a known **grid topology**: nearby pixels are strongly related
- Useful features are **local**: an edge depends on a few neighbouring pixels
- Useful features are **translation-invariant**: an edge looks the same anywhere in the image

Convolutional networks exploit all three facts. They replace dense matrix multiplication with **convolution** in at least one layer.

The Convolution Operation

Discrete convolution (1D)

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)\, w(t-a)$$

- x: the **input**
- w: the **kernel** (learned parameters)
- s: the output, or **feature map**

Two dimensions (images)

$$S(i,j) = (I*K)(i,j) = \sum_{m} \sum_{n} I(i+m, j+n)\, K(m,n)$$

(In practice, most libraries use cross-correlation (no kernel flipping) but still call it convolution.)

Key idea:

Slide a small kernel across the input. At each position, compute a weighted sum of the local input patch. The same kernel weights are used at every position. **What a kernel does:**

- A small kernel detects one local pattern (an edge, a colour blob, a texture)
- The output feature map shows *where* that pattern appears in the input
- Many kernels in parallel detect many different features

Connection to linear algebra: convolution is matrix multiplication by a sparse, structured (Toeplitz) matrix with many tied entries.

Convolution: A 2D Example

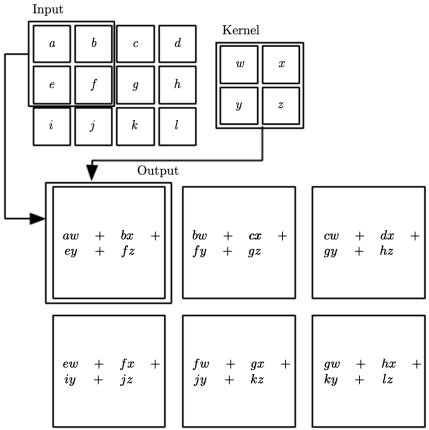

Figure: 2D convolution without kernel flipping.

Reading the figure:

- The kernel slides over the input image
- Each output element is the weighted sum of the input patch currently under the kernel
- "Valid" convolution: the kernel only visits positions where it fits entirely inside the input
- The output is smaller than the input (by kernel width -1 in each dimension)

Same kernel everywhere:

Notice that the *same* kernel weights (w, x, y, z) are reused to produce every output element. This is parameter sharing, which we examine next.

Motivation 1: Sparse Interactions

Dense layers: full interaction

Every output unit connects to every input unit. With m inputs and n outputs:

- $m \times n$ parameters
- $O(m \times n)$ runtime per example

Convolution: sparse interaction

The kernel is much smaller than the input. If each output connects to only k inputs:

- $k \times n$ parameters
- $O(k \times n)$ runtime per example
- k can be thousands of times smaller than m

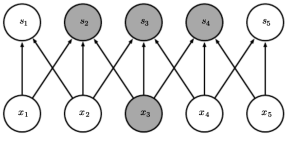

Figure: Sparse connectivity in convolutional networks: each output unit depends only on a small subset of input units, known as its receptive field.

Deep receptive fields: Although each connection is local, units in *deeper* layers indirectly depend on a large portion of the input. Stacking convolutions grows the effective receptive field.

Motivation 2: Parameter Sharing

Parameter sharing (tied weights)

The same kernel parameter is used at *every* position of the input, rather than learning a separate weight for each location.

In a dense layer: each weight is used exactly once.

In a convolutional layer: each kernel weight is reused at every spatial position. **Consequences:**

- Storage drops to just k parameters per kernel (not $k \times n$)
- The model learns one feature detector that works everywhere
- Dramatically improved statistical efficiency

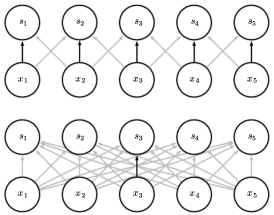

Figure: Parameter sharing: a single kernel is applied at every input location, greatly reducing the number of learnable parameters.

Example: edge detection

A two-element kernel that subtracts neighbouring pixels detects vertical edges across an entire image. Describing the same operation with a dense matrix would need billions of entries, convolution is millions of times more efficient.

Motivation 3: Equivariance to Translation

Equivariance

A function f is **equivariant** to a transformation g if

$$f(g(x)) = g(f(x)).$$

Convolution is equivariant to translation: shifting the input shifts the output by the same amount.

Why this matters:

- If an object moves in the image, its feature-map representation moves the same way
- The network does not need to relearn the feature at the new location
- Perfect for features that are useful everywhere (edges, textures)

What convolution is not equivariant to:

- Changes in **scale** (object larger/smaller)
- **Rotation** of the image
- Other geometric transformations

These require separate mechanisms (e.g. data augmentation, multi-scale processing).

When sharing is undesirable:

Sometimes we want *different* features at different locations. For face images cropped and centred, the top should detect eyebrows and the bottom a chin — here locally connected layers (unshared weights) may be preferable.

Pooling: Definition and Purpose

Pooling

A pooling function replaces the output at a location with a **summary statistic** of nearby outputs.

Common pooling functions:

- **Max pooling:** the maximum of a rectangular neighbourhood
- Average of a neighbourhood
- L^2 norm of a neighbourhood
- Weighted average by distance from the centre

The three stages of a typical conv layer:

1. Convolution stage (affine transform)
2. Detector stage (nonlinearity, e.g. ReLU)
3. Pooling stage

Why pool? Local translation invariance.

- If the input shifts by a small amount, most pooled outputs do not change
- Useful when we care *whether* a feature is present, not exactly *where*

Example: face detection

To know an image contains a face, we need to know there is an eye on the left and an eye on the right — not their pixel-perfect positions. Pooling discards precise location and keeps presence.

Pooling: Translation Invariance

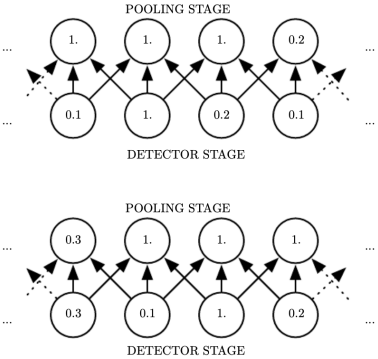

Figure: Max pooling provides local translation invariance: small input shifts alter detector outputs but leave most pooled outputs unchanged.

Reading the figure:

- The bottom row shows detector (nonlinearity) outputs
- The top row shows max-pooled outputs over a width-3 region
- After shifting the input by one pixel, every *detector* value changes
- But only half the *pooled* values change, max pooling is insensitive to small shifts

Pooling as an infinitely strong prior:

Using pooling assumes the layer's function *must* be invariant to small translations. When this assumption is correct, it greatly improves statistical efficiency.

Pooling: Downsampling and Variable-Size Inputs

Downsampling with pooling:

- Report pooling summaries spaced k pixels apart instead of every pixel
- The next layer has roughly k times fewer inputs
- Improves computational and statistical efficiency

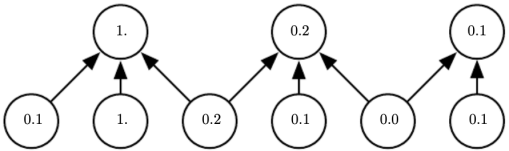

Figure: Max pooling provides local translation invariance: small input shifts alter detector outputs but leave most pooled outputs unchanged.

Handling variable-size inputs:

Many classifiers require a fixed-size input to the final dense layer. But images come in different sizes.

Solution: use pooling regions whose *size* scales with the input, but whose *number* is fixed.

- E.g. always output 4 summary statistics, one per image quadrant
- The dense classifier then always sees the same number of features

Pooling lets a CNN process images of varying size, something a fixed dense matrix cannot do.

Convolution Variants: Stride and Zero-Padding

Stride:

Skip positions of the kernel to downsample directly during convolution. A stride of s samples every s-th position, reducing output size by a factor of s in each direction.

Zero-padding:

Implicitly pad the input with zeros so we can control the output size independently of the kernel size. Without padding, the representation shrinks at every layer.

Three padding regimes:

- **Valid:** no padding. Kernel stays fully inside the input. Output shrinks by $k - 1$ each layer. Limits network depth.
- **Same:** pad just enough to keep output size = input size. Allows arbitrarily deep networks.
- **Full:** pad so every input pixel is visited k times. Output *grows* by $k - 1$.

The optimal padding usually lies between "valid" and "same."

Locally connected layers: same connectivity as convolution but *without* sharing, each location has its own weights. Useful when the same feature should not recur everywhere.

Full CNN Architectures

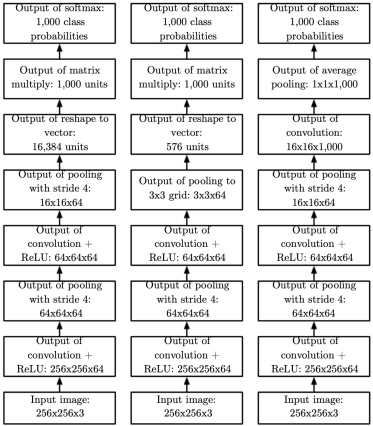

Figure: CNNs alternate convolution, nonlinearity, and pooling before classification.

Typical structure:

1. Alternate convolution → ReLU → pooling for several layers
2. Spatial dimensions shrink; channel depth grows
3. Flatten the final feature map
4. One or more dense layers
5. Softmax over classes

Three design variants (figure):

- **Left:** fixed image size → flatten → dense classifier
- **Centre:** variable image size → variably-sized pooling → fixed-size vector → dense
- **Right:** fully convolutional, one feature map per class, global average pooling, no dense layer

Convolution and Pooling as an Infinitely Strong Prior

A prior, expressed as architecture:

A CNN is like a dense network with an *infinitely strong prior* on its weights:

- **Convolution prior:** the weights for a unit must equal those of its neighbour, shifted in space, and must be zero outside a small receptive field. The layer learns only **local, translation-equivariant** interactions.
- **Pooling prior:** each unit must be **invariant to small translations**.

Key insight: priors can cause underfitting

Like any prior, convolution and pooling help *only* when their assumptions hold.

- If a task needs precise spatial information, pooling everywhere can hurt
- If a task needs to combine very distant input locations, the locality prior is wrong

Fair benchmarking

Only compare convolutional models to other convolutional models. Permutation-invariant models (which would work even if pixels were shuffled) must discover spatial structure on their own.

Neuroscience Basis and Module 8 Summary

Inspired by the visual cortex (V1):

- V1 is arranged in a 2D spatial map mirroring the retina $\rightarrow$ feature maps
- **Simple cells** respond to local, oriented patterns $\rightarrow$ detector units
- **Complex cells** respond to the same features but are invariant to small shifts $\rightarrow$ pooling units
- Reverse-correlation shows V1 weights resemble **Gabor functions**; oriented edge detectors
- Many learning algorithms recover Gabor-like filters in their first layer

Module 8 summary:

- **Convolution** replaces dense multiplication, giving sparse interactions, parameter sharing, and translation equivariance
- **Pooling** summarises neighbourhoods, giving local translation invariance and variable-input handling
- A typical layer = convolution $\rightarrow$ nonlinearity $\rightarrow$ pooling
- Architectures stack these, shrinking space and growing depth, then classify
- CNNs encode strong priors that match the structure of images

Why Sequences Need Special Treatment

Sequential data is everywhere:

- Text (sequences of words or characters)
- Speech and audio (sequences over time)
- Time series (sensor readings, prices)
- Video (sequences of frames)

The problem with fixed-size models:

- Sequences have **variable length**
- A fixed-size dense network cannot accept inputs of different lengths
- The same information can appear at **different positions**

The position problem

"I went to Nepal in 2009" vs. "In 2009, I went to Nepal."

A model that should extract the travel year must recognise "2009" whether it appears in the second word or the sixth.

A dense feedforward network would learn separate parameters for each position — it would have to relearn the meaning of "2009" independently at every slot.

The solution: share parameters across positions, so the same rule applies wherever the information appears.

Parameter Sharing Across Time

The core idea:

Apply the *same* transition function with the *same* parameters at every time step.

A dynamical system

$$h^{(t)} = f\left(h^{(t-1)}, x^{(t)}; \theta\right)$$

- $h^{(t)}$: the hidden state (a lossy summary of the past)
- $x^{(t)}$: the input at time t
- θ: shared parameters, identical at every step

Why parameter sharing is powerful:

1. **Variable length:** the model is defined by a transition from one state to the next, not by a fixed-length history, so it handles any sequence length
2. **Generalisation:** a pattern learned at one position transfers to all positions
3. **Fewer parameters:** one shared function instead of one per time step

Contrast with 1D convolution:

Convolution also shares parameters across time, but is *shallow*, each output depends on a few neighbours. RNNs share through a *deep* chain, where each output depends on the entire past.

Unfolding the Computational Graph

From recurrence to a deep graph:

The recurrent definition

$$h^{(t)} = f\left(h^{(t-1)}, x^{(t)}; \theta\right)$$

can be **unfolded** by repeatedly substituting itself:

$$h^{(3)} = f\left(f\left(h^{(1)}, x^{(2)}; \theta\right), x^{(3)}; \theta\right)$$

The result is a directed acyclic graph with one node per time step, and the same θ used everywhere.

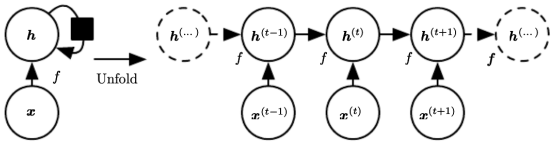

Figure: Recurrent networks can be unfolded over time into a feedforward computation graph.

Two advantages of unfolding:

1. The learned model always has the same input size (a state-to-state transition), regardless of sequence length
2. The same transition function f with the same θ is reused at every step

The Recurrent Neural Network

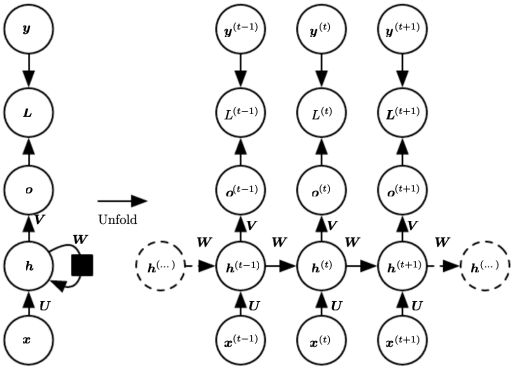

Figure: Recurrent networks use input-to-hidden, hidden-to-hidden, and hidden-to-output weight matrices across time steps.

Three weight matrices (all shared across time):

- U: input-to-hidden connections
- W: hidden-to-hidden (recurrent) connections
- V: hidden-to-output connections

Information flow:

- At each step, the hidden state combines the previous state (via W) with the new input (via U)
- The output is read from the hidden state (via V)
- The hidden state carries information forward through time

This RNN is **universal**: with enough units it can compute any function a Turing machine can.

RNN: Forward Propagation Equations

For each time step $t = 1, \ldots, \tau$:

$$a^{(t)} = b + Wh^{(t-1)} + Ux^{(t)}$$

$$h^{(t)} = \tanh\left(a^{(t)}\right)$$

$$o^{(t)} = c + Vh^{(t)}$$

$$\hat{y}^{(t)} = \text{softmax}\left(o^{(t)}\right)$$

- b, c: bias vectors
- $a^{(t)}$: pre-activation
- $h^{(t)}$: hidden state (uses tanh)
- $o^{(t)}$: output logits
- $\hat{y}^{(t)}$: predicted probability distribution

Reading the equations:

- The hidden state $h^{(t)}$ depends on *both* the previous hidden state and the current input
- A tanh nonlinearity squashes the hidden activation
- The output is a softmax over the vocabulary (for word/character prediction)

Initial state:

Forward propagation begins with a specified initial hidden state $h^{(0)}$ (often zero), then proceeds left to right through the sequence.

This RNN maps an input sequence to an output sequence of the **same length**, one prediction per time step.

RNN: Loss and Back-Propagation Through Time

The total loss is the sum of per-step losses:

$$L = \sum_t L^{(t)} = -\sum_t \log p_{\text{model}}\left(y^{(t)} \mid x^{(1)}, \ldots, x^{(t)}\right)$$

Back-Propagation Through Time (BPTT):

- Apply standard backpropagation to the *unfolded* graph
- A forward pass left to right, then a backward pass right to left
- No specialised algorithm is needed

Cost of BPTT:

- **Runtime** is $O(\tau)$ and **inherently sequential**, each step depends on the previous one, so it cannot be parallelised across time
- **Memory** is $O(\tau)$, all hidden states from the forward pass must be stored for the backward pass

Gradient w.r.t. shared parameters:

Because U, V, W are shared across all time steps, their gradients are *summed* over every step in which they are used.

The hidden-to-hidden recurrence makes the RNN powerful, but BPTT makes it expensive to train on long sequences.

RNN Design Patterns

Common RNN configurations:

1. **Output at each step, hidden-to-hidden recurrence** (the canonical RNN). Most powerful; can simulate a Turing machine.
2. **Output at each step, output-to-hidden recurrence only.** Less powerful (output must capture all relevant past), but each step can be trained in isolation $\Rightarrow$ parallelisable.
3. **Read entire sequence, single output at the end.** Used to summarise a sequence into a fixed-size vector (e.g. sentiment classification).

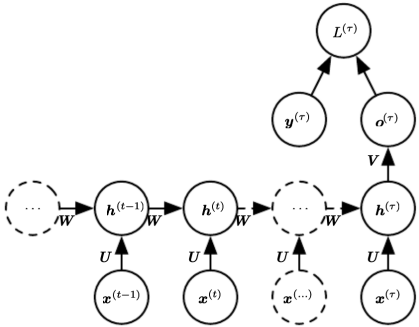

Figure: An RNN processes a full sequence and produces a single fixed-size representation at the final time step.

Choosing a pattern: The right pattern depends on the task: per-step outputs for tagging or language modelling, single output for classification, and so on.

Teacher Forcing

Teacher forcing

During training, feed the **ground-truth** output $y^{(t)}$ (not the model's own prediction) as the input at time $t+1$.

Why:

It emerges from the maximum likelihood criterion. For models with output-to-hidden recurrence, it lets every time step be trained in isolation, avoiding BPTT and allowing parallel training.

During training: feed the correct $y^{(t)}$.

At test time: the true output is unknown, so feed the model's own output back in.

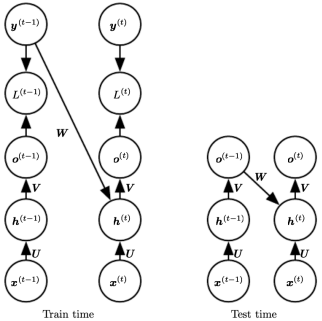

Figure: Teacher forcing feeds the correct output during training, but uses the model's own predictions at test time.

The exposure-bias problem

Training only on correct inputs causes test-time mismatch; mixing true and predicted inputs helps.

Bidirectional RNNs (1/2)

Limitation of causal RNNs:

A standard RNN state at time t captures only the *past* $(x^{(1)}, \ldots, x^{(t)})$, but many tasks require **future context**.

Why future context matters

In speech recognition, the correct phoneme may depend on upcoming sounds or words (co-articulation effects).

Idea: incorporate information from both past and future.

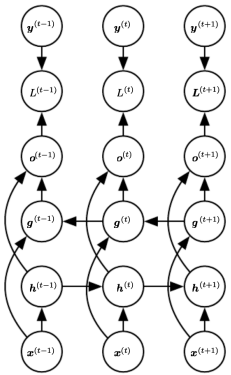

Figure: Bidirectional RNNs combine forward and backward recurrences so each output depends on both past and future context.

Bidirectional RNNs (2/2)

Architecture:

- Forward RNN processes sequence left to right
- Backward RNN processes sequence right to left
- Outputs are based on both hidden states

Key idea: Each output $o^{(t)}$ depends on:

- past context (forward RNN)
- future context (backward RNN)

Interpretation: The model uses the full sequence to build a richer representation centered at time t.

Limitation: Requires the full sequence in advance $\rightarrow$ not suitable for real-time streaming tasks.

Encoder–Decoder (Sequence-to-Sequence) (1/2)

The problem:

Map an input sequence to an output sequence of a *different* length (e.g. machine translation, question answering).

Encoder–decoder idea

- An **encoder** RNN reads the input sequence
- It produces a fixed-size **context vector** C
- A **decoder** RNN is conditioned on C

The model learns:

$$\log P(y \mid x)$$

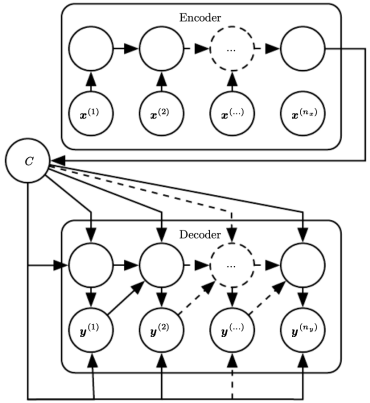

Figure: Encoder–decoder architectures use an encoder RNN to compress the input into a context vector, which a decoder RNN uses to generate the output sequence.

Encoder–Decoder (Sequence-to-Sequence) (2/2)

Sequence modeling setup:

- Input length n_x, output length n_y
- Encoder and decoder trained jointly

Objective:

$$\log P(y^{(1)}, \ldots, y^{(n_y)} \mid x^{(1)}, \ldots, x^{(n_x)})$$

Key idea: A single vector C summarizes the entire input sequence.

Limitation

If C must compress a long sequence, important information may be lost. This bottleneck motivated **attention mechanisms**, where the decoder can access all encoder states.

The Challenge of Long-Term Dependencies

The core difficulty:

Gradients propagated over many time steps tend to either **vanish** (usually) or **explode** (rarely, but destructively).

Why: repeated multiplication.

Consider a simplified linear recurrence $h^{(t)} = Wh^{(t-1)}$. After t steps:

$$h^{(t)} = W^t h^{(0)}$$

With eigendecomposition $W = Q\Lambda Q^\top$:

$$h^{(t)} = Q\Lambda^t Q^\top h^{(0)}$$

Eigenvalues are raised to the power t.

The consequence:

- Eigenvalues with magnitude < 1 **decay to zero** (vanishing)
- Eigenvalues with magnitude > 1 **grow without bound** (exploding)

Why this is fundamental:

To store memories robustly, the RNN must operate in a regime where gradients vanish. Long-term interactions receive *exponentially smaller* gradients than short-term ones.

In practice

Traditional RNNs trained with SGD struggle to learn dependencies spanning more than 10–20 steps. This is one of the central challenges in deep learning.

Gated RNNs: The LSTM

The idea behind gating:

Create paths through time where gradients neither vanish nor explode, by letting the network *learn* when to remember and when to forget.

The LSTM cell (Hochreiter & Schmidhuber, 1997) introduces:

- A **state unit** with a linear self-loop (the cell state)
- A **forget gate** controlling how much of the old state to keep
- An **input gate** controlling how much new information to add
- An **output gate** controlling what to expose

The self-loop weight is *gated* (controlled by context), so the time scale of integration changes dynamically.

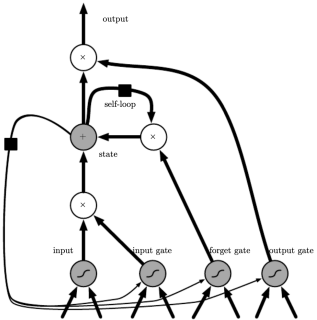

Figure: LSTM cell: a memory unit with a linear self-loop controlled by input, forget, and output gates.

Why it helps:

When the forget gate is near 1, the cell state is preserved and gradients flow without vanishing.

LSTM: The Gating Equations

Forget gate (how much old state to keep):

$$f_i^{(t)} = \sigma\Big(b_i^f + \sum_j U_{ij}^f x_j^{(t)} + \sum_j W_{ij}^f h_j^{(t-1)}\Big)$$

Cell state update (conditional self-loop):

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + g_i^{(t)}\, \sigma\Big(b_i + \sum_j U_{ij} x_j^{(t)} + \sum_j W_{ij} h_j^{(t-1)}\Big)$$

Input (external) gate:

$$g_i^{(t)} = \sigma\Big(b_i^g + \sum_j U_{ij}^g x_j^{(t)} + \sum_j W_{ij}^g h_j^{(t-1)}\Big)$$

Output gate and **cell output:**

$$q_i^{(t)} = \sigma\Big(b_i^o + \sum_j U_{ij}^o x_j^{(t)} + \sum_j W_{ij}^o h_j^{(t-1)}\Big)$$

$$h_i^{(t)} = \tanh\Big(s_i^{(t)}\Big)\, q_i^{(t)}$$

Reading the equations:

- Each gate is a sigmoid (output in $[0, 1]$) with its own weights
- The forget gate f multiplies the old cell state
- The input gate g controls new information entering the state
- The output gate q controls what the cell exposes

LSTMs learn long-term dependencies far more reliably than plain RNNs.

The Gated Recurrent Unit (GRU)

A simpler gated RNN (Cho et al., 2014):

The GRU uses a *single* gate to control both forgetting and updating, with fewer parameters than the LSTM.

State update:

$$h_i^{(t)} = u_i^{(t-1)} h_i^{(t-1)} + (1 - u_i^{(t-1)})\, \sigma\Big(\cdots \Big)$$

where the inner term mixes the input and the *reset*-gated previous state.

Two gates:

- **Update gate** u: how much of the old state to carry forward vs. replace
- **Reset gate** r: how much of the past state to use when computing the new candidate state

Update and reset gates:

$$u_i^{(t)} = \sigma\Big(b_i^u + \textstyle\sum_j U_{ij}^u x_j^{(t)} + \sum_j W_{ij}^u h_j^{(t)} \Big)$$

$$r_i^{(t)} = \sigma\Big(b_i^r + \textstyle\sum_j U_{ij}^r x_j^{(t)} + \sum_j W_{ij}^r h_j^{(t)} \Big)$$

LSTM vs. GRU:

- GRU has fewer gates and parameters
- Often comparable performance
- No single variant consistently beats both across all tasks
- A key ingredient is the forget gate; setting its bias to 1 helps the LSTM

Gradient Clipping and Module 9 Summary

Gradient clipping (for exploding gradients):

If the gradient norm exceeds a threshold v, rescale before updating:

$$\text{if } \|g\| > v: \quad g \leftarrow \frac{g\, v}{\|g\|}$$

- Preserves the gradient *direction*
- Bounds the step size so a cliff cannot catapult the parameters away
- Handles exploding gradients; does *not* help vanishing gradients (use LSTMs/GRUs for that)

Module 9 summary:

- RNNs process sequences by **sharing parameters across time**
- The hidden state is a lossy summary of the past; BPTT trains it at $O(\tau)$ sequential cost
- Patterns: per-step output, single output, encoder–decoder, bidirectional
- **Long-term dependencies** are hard: gradients vanish or explode over many steps
- **LSTMs and GRUs** use gating to create stable gradient paths
- Gradient clipping tames exploding gradients

RNNs were the dominant sequence models, until attention and transformers (Module 10).

Module 10: Attention and Transformers

Supplementary material

The Goodfellow textbook predates the transformer revolution. This module draws on Jurafsky & Martin (2024), *Speech and Language Processing*.

Why transformers matter:

- They are the architecture behind modern large language models
- They power machine translation, summarisation, question answering, and chatbots
- They replaced RNNs as the dominant sequence model

What we will cover:

- The **self-attention** mechanism
- Query, key, and value projections
- **Multi-head** attention
- The **transformer block**
- Token and position embeddings
- The language modelling head
- Generation by sampling
- Training, scaling laws, and potential harms

Learning outcome: understand the conceptual foundations behind modern AI systems.

The Limitation of RNNs

RNNs process sequences serially:

- Each hidden state depends on the previous one
- Computation **cannot be parallelised** across time
- Training is slow on long sequences

RNNs struggle with distant information:

- Information must pass through every intermediate state
- Gradients vanish over long distances
- Even LSTMs have a practical limit on how far back they reliably reach

What we want instead:

- A mechanism that lets any position **directly** access any other position — no matter how far apart
- Computation that can be **parallelised** across all positions at once

Self-attention

The transformer's key innovation. It allows a network to directly extract and combine information from arbitrarily large contexts, while remaining fully parallelisable.

The Intuition of Self-Attention (1/2)

Contextual representations:

Transformers build increasingly rich representations of each word across layers.
At each layer, representation of word *i* is formed by combining:

- Its previous-layer representation
- A weighted combination of all other words

Key idea: relevance is learned via attention weights.

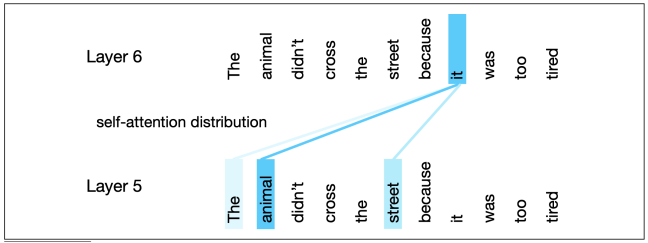

Figure: Self-attention shows how each token weights other tokens when forming its representation.

The Intuition of Self-Attention (2/2)

Why context matters:

- "The keys to the cabinet *are* on the table" $\rightarrow$ subject-verb agreement
- "The chicken crossed the road because *it* was tired" $\rightarrow$ pronoun resolution
- "The trees along the *bank*" $\rightarrow$ word sense disambiguation

Core mechanism: Each word can attend to *all other words* in the sequence, regardless of distance.

Insight: Self-attention removes the notion of locality and enables global interactions in a single layer.

Attention: Comparing Words

The core operation: a comparison.

Attention compares an item of interest to a collection of other items, revealing their relevance in the current context.

How to compare two word vectors?

Use the **dot product**, a scalar measure of similarity:

$$\text{score}(x_i, x_j) = x_i \cdot x_j$$

- Larger value $\Rightarrow$ more similar vectors
- Ranges from $-\infty$ to $+\infty$

From scores to weights:

Normalise the scores with a softmax to get attention weights that sum to 1:

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j)), \quad j \leq i$$

From weights to output:

The output for position i is the weighted sum of the context:

$$a_i = \sum_{j \leq i} \alpha_{ij} x_j$$

Compare $\rightarrow$ normalise $\rightarrow$ weighted sum.
This is the heart of attention.

Query, Key, and Value

Three roles each word plays:

- **Query** (q): the current word, being compared against all others
- **Key** (k): a word being compared *against* the query
- **Value** (v): the content actually combined to form the output

Learned projections:

Three weight matrices project each input into its three roles:

$$q_i = x_i W^Q, \quad k_i = x_i W^K, \quad v_i = x_i W^V$$

Why separate roles?

A word's usefulness *as a query* (what it is looking for) differs from its usefulness *as a key* (what it offers to others) and *as a value* (the information it contributes).

Dimensions:

- Input/output dimension: d
- Key and query dimension: d_k
- Value dimension: d_v
- In the original transformer: $d = 512$, $d_k = d_v = 64$

Separating query, key, and value gives the model far more expressive power than raw dot products of word vectors.

Self-Attention: The Full Computation (1/2)

For a single output a_i:

$$q_i = x_i W^Q, \quad k_i = x_i W^K, \quad v_i = x_i W^V$$

Scaled similarity:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Attention weights (causal):

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j)), \quad j \leq i$$

Output representation:

$$a_i = \sum_{j \leq i} \alpha_{ij} \, v_j$$

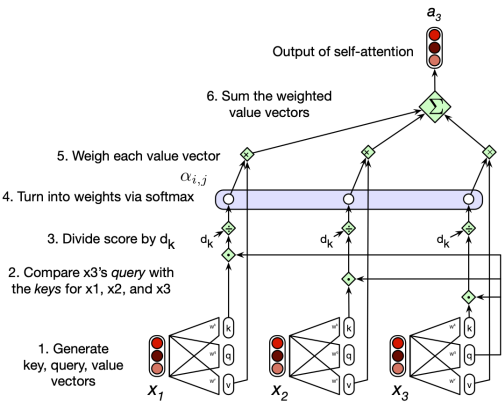

Figure: Causal self-attention computes each output as a weighted sum of value vectors, using softmax-normalized similarity between query and key vectors.

Self-Attention: The Full Computation (2/2)

Why scale by $\sqrt{d_k}$?

- Dot products grow with dimension
- Large values push softmax into saturation
- Gradients become very small

Effect of scaling: Stabilizes training by keeping attention scores in a well-conditioned range.

Self-Attention: Matrix Form

Parallelising across all positions:

Pack the N input embeddings into a matrix $X \in \mathbb{R}^{N \times d}$ (one row per token). Then:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

The entire self-attention layer in one expression:

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

What each piece does:

- $QK^\top$: all query-key comparisons at once, an $N \times N$ matrix
- Scale by $\sqrt{d_k}$, then softmax each row
- Multiply by V: weighted sum of value vectors
- Output: $N \times d$, one contextual vector per token

Cost: quadratic in length

Self-attention computes dot products between every pair of tokens, so cost grows as $O(N^2)$. This makes very long inputs (entire novels) expensive, though modern models handle thousands of tokens.

Masking Out the Future

The problem with language modelling:

When predicting the next word, the model must *not* see the words that follow; otherwise, the task is trivial (the answer is right there).

Causal (backward-looking) self-attention:

- Each position may attend only to itself and *earlier* positions
- The upper triangle of the $QK^\top$ matrix is set to $-\infty$
- After softmax, those entries become 0

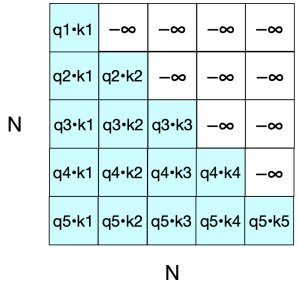

Figure: Causal masking prevents each token from attending to future tokens.

Two kinds of attention:

- **Causal:** context is the prior words only (used for generation)
- **Bidirectional:** context includes future words (used in models like BERT)

Multi-Head Attention (1/2)

Why one attention head is not enough:

Words can relate in many ways simultaneously; syntactic, semantic, and referential. A single attention computation may not capture all of these relationships effectively.

Multi-head attention

Run several attention computations ("heads") in parallel, each with its own projection matrices:

$$W_i^Q, \quad W_i^K, \quad W_i^V.$$

Each head can learn a different type of relationship.

$$\text{head}_i = \text{SelfAttention}(XW_i^Q,\ XW_i^K,\ XW_i^V)$$

$$A = (\text{head}_1 \oplus \cdots \oplus \text{head}_h)\, W^O$$

Multi-Head Attention (2/2)

How it fits together:

- Each of h heads produces an $N \times d_v$ output
- Outputs are concatenated into an $N \times hd_v$ matrix
- A final projection W^O maps back to dimension d

Original Transformer:

$$h = 8, \qquad d_k = d_v = 64, \qquad d = 512$$

Benefit: Different heads can focus on different linguistic relationships simultaneously.

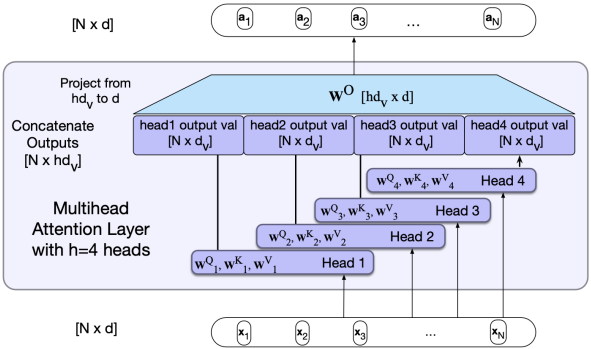

Figure: Multi-head attention uses multiple attention mechanisms in parallel to capture different relationships in the input.

The Transformer Block (1/2)

A transformer block combines four components:

1. Multi-head self-attention
2. Feedforward network (FFN)
3. Residual connections
4. Layer normalization

Post-norm formulation:

$$O = \text{LayerNorm}(X + \text{SelfAttention}(X))$$
$$H = \text{LayerNorm}(O + \text{FFN}(O))$$

These operations are stacked repeatedly to build deep transformer models.

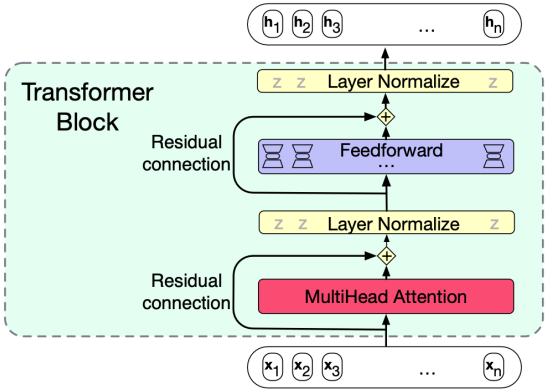

Figure: A transformer block combines multi-head attention and a feedforward network with residual connections and layer normalization.

The Residual Stream View

An alternative way to see the block:

Follow a *single token's* vector as it flows up through the layers — the **residual stream**.

- The input embedding enters at the bottom
- Residual connections copy it upward unchanged
- Attention and feedforward layers *add* new information into the stream

Pre-norm variant (often trains better): layer norm is applied *before* each sublayer rather than after:

$$t_i^1 = \text{MultiHeadAttn}(\text{LayerNorm}(x_i), \ldots)$$

$$h_i = \text{FFN}(\text{LayerNorm}(\cdots)) + \cdots$$

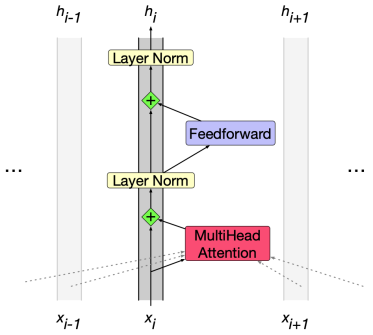

Figure: How a token evolves through the residual stream.

Key observation:

Only attention mixes information across tokens; FFN and LayerNorm act per token.

Input: Token and Position Embeddings

Two embeddings are combined:

1. **Token embedding:** look up each token's vector in the embedding matrix E (shape $|V| \times d$)
2. **Position embedding:** a vector encoding *where* the token sits in the sequence

The input representation is their sum:

$$X[i] = E[\text{token}_i] + P[i]$$

Both are $1 \times d$, so their sum is too.

Why position embeddings?

Self-attention has no inherent notion of order; it treats the input as a set. Position embeddings inject the sequence order.

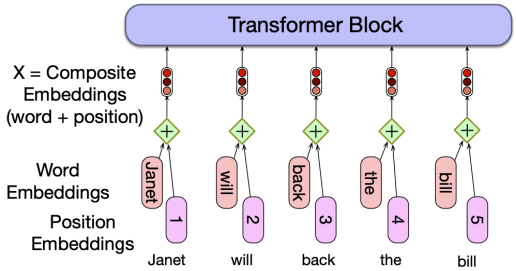

Figure: Token and position embeddings are added to encode identity and order.

Two approaches to position:

- **Absolute:** learned embeddings, one per position
- **Static functions** (sine/cosine of different frequencies) or **relative** position methods, which can generalize better to unseen lengths

The Language Modeling Head

From hidden state to next-word probabilities:

The job of the language modelling head is to turn the final-layer output for the last token into a distribution over the vocabulary.

Two steps:

$$u = h_N^L E^\top \qquad \text{(logits, } 1 \times |V|\text{)}$$

$$y = \text{softmax}(u) \qquad \text{(probabilities)}$$

Weight tying: the unembedding matrix is the *transpose* of the input embedding matrix E. The same E maps tokens in and maps hidden states back out.

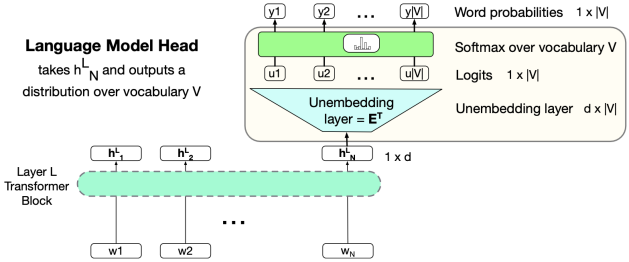

Figure: Final token representation is projected to vocabulary logits via a linear (unembedding) layer and softmax.

Generating a word:

Whatever entry y_k we choose from the probability vector, we emit the word with index k. *How* we choose (greedy vs. sampling) is the decoding strategy.

Stacking Blocks: The Full Architecture

A decoder-only language model:

1. Convert input tokens to embeddings; add position embeddings
2. Pass through a stack of transformer blocks (12 to 96+ layers)
3. Apply the language modelling head to the final token
4. Produce a probability distribution over the next word

Input/output consistency

Each block preserves shape ($N \times d$), allowing deep stacking.

Why "decoder-only"?

This is the decoder half of the original encoder–decoder transformer, used alone for causal language modeling.

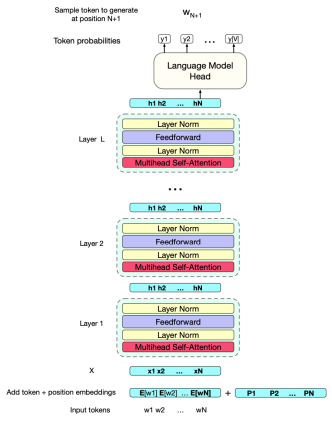

Figure: The full decoder-only transformer: stacked blocks map input tokens $w_1, \ldots, w_N$ to a predicted next token w_{N+1}.

Stacking Blocks: Scale and Interpretation

Key property: modular stacking

- Each transformer block preserves representation size ($N \times d$)
- Enables deep architectures with dozens to hundreds of layers

Why scaling works

- More layers $\rightarrow$ more expressive hierarchical computation
- Larger context $\rightarrow$ richer dependency modeling

Scale of modern models

- Context windows: 1K–4K+ tokens (and growing)
- Depth: 12 to 100+ layers
- Parameters: billions to hundreds of billions

Large Language Models: Tasks as Word Prediction (1/2)

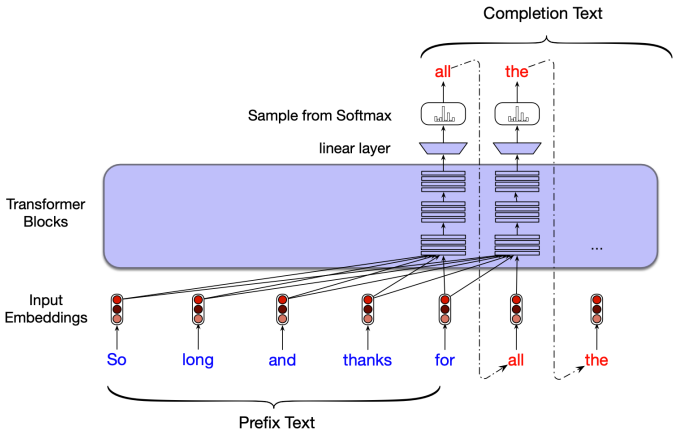

Figure: Autoregressive generation: each token is predicted from the prompt and all previously generated tokens.

Large Language Models: Tasks as Word Prediction (2/2)

Central insight

Many NLP tasks can be reformulated as next-word prediction given a suitable prompt.
This is called **conditional generation**.

Sentiment analysis as word prediction

Prompt: `The sentiment of "I like Jackie Chan" is:`
Compare $P(\texttt{positive} \mid \text{prompt})$ vs. $P(\texttt{negative} \mid \text{prompt})$.

Question answering

Prompt: `Q: Who wrote The Origin of Species?`
`A:` High probability is assigned to `Charles Darwin`.

Why transformers excel

Long context windows allow attention over the entire prompt and generated history, enabling strong performance on summarization, translation, and dialogue.

Generation: Greedy Decoding and Sampling

Greedy decoding:

At each step, emit the single most probable word:

$$\hat{w}_t = \arg\max_{w \in V} P(w \mid w_{<t})$$

- Simple and deterministic
- But produces generic, repetitive text
- Rarely used for open-ended generation

Random sampling:

Choose the next word randomly according to its probability. More diverse, but the long tail of low-probability words occasionally produces strange output.

Quality vs. diversity trade-off:

- Emphasising the **most probable** words: more accurate, coherent, factual; but boring and repetitive
- Giving weight to **middle-probability** words: more creative and diverse; but less factual and sometimes incoherent

Autoregressive generation

Decode left to right, each word conditioned on all previous choices, until an end-of-sequence token is produced.

Generation: Top-k, Top-p, and Temperature

Top-k sampling:

- Keep only the k most probable words
- Renormalise and sample from them
- $k = 1$ is greedy decoding; larger k adds diversity
- Limitation: k is fixed regardless of how peaked the distribution is

Top-p (nucleus) sampling:

- Keep the smallest set of words whose cumulative probability $\geq p$

$$\sum_{w \in V^{(p)}} P(w \mid w_{<t}) \geq p$$

- The candidate pool grows and shrinks with the context

Temperature sampling:

Reshape (don't truncate) the distribution by dividing the logits by a temperature τ before softmax:

$$y = \text{softmax}(u/\tau)$$

- Low τ ($\in (0, 1]$): sharpens toward high-probability words (more greedy)
- As $\tau \to 0$: approaches greedy decoding
- $\tau > 1$: flattens the distribution (more diverse)

These methods all trade off quality against diversity. The right choice depends on the task.

Training Transformers: Self-Supervision (1/2)

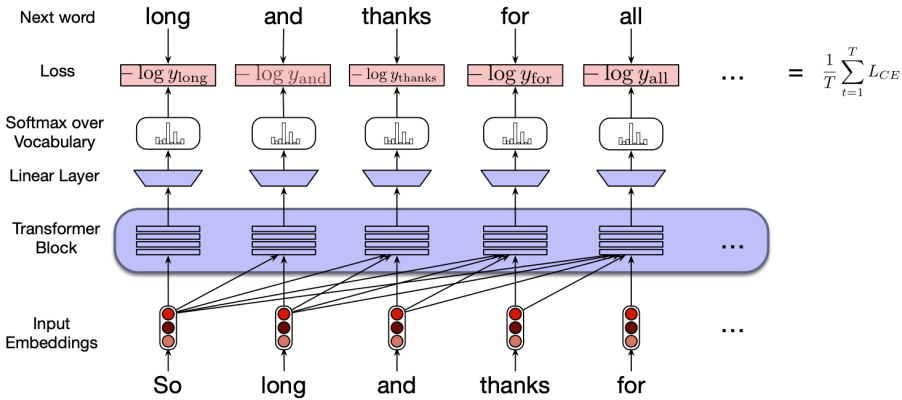

Figure: Transformer training: at each position the model predicts the next word, and cross-entropy loss is computed against the true next word.

Training Transformers: Self-Supervision (2/2)

Self-supervised objective

Train on large text corpora by predicting the next word at each position.
The text itself provides the supervision.

Cross-entropy loss

$$L_{CE} = -\log \hat{y}_t[w_{t+1}]$$

Sequence loss is averaged over all positions.

Teacher forcing

Always condition on the *true* previous tokens during training.

Parallelism advantage over RNNs

All positions in a sequence are processed in parallel, unlike the sequential computation in RNNs.

Training data

Web-scale corpora (e.g. Common Crawl), books, and Wikipedia.
Training uses very large batches (millions of tokens).

Scaling Laws

Three factors determine LLM performance:

1. Model size N (number of parameters)
2. Dataset size D (training tokens)
3. Compute budget C

Power-law relationships:

When the other two factors are not limiting, the loss falls as a power law in each:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}$$

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

(Kaplan et al., 2020)

Estimating parameter count:

Ignoring biases, with d the model dimension and n_{layer} the number of layers:

$$N \approx 12\, n_{\text{layer}}\, d^2$$

Example: GPT-3 with $n_{\text{layer}} = 96$ and $d = 12288$ gives roughly 1.75×10^{11} (175 billion) parameters.

Why scaling laws matter:

- Predict performance before training a huge model
- Decide how to allocate compute between bigger models and more data
- Improve a model by adding parameters, data, or training time

Potential Harms from Language Models

Hallucination:

Models are trained to produce *predictable, coherent* text; not necessarily *true* text. They can state falsehoods confidently. A serious problem wherever facts matter.

Toxicity and bias:

- Even benign prompts can elicit hate speech or abuse
- Models reproduce and can amplify stereotypes and negative attitudes
- Training data is disproportionately from certain groups and regions, skewing outputs

Misinformation and misuse:

- Can generate misinformation, phishing, or extremist content at scale

Privacy and copyright:

- Models can leak memorised training data (names, addresses, phone numbers)
- Trained on copyrighted text, raising fair-use questions

Mitigation

Carefully analyse and document training data (datasheets, model cards). Transparency about training corpora is increasingly required by regulation and is essential for understanding toxicity, bias, privacy, and fair use.

Module 10 Summary: Attention and Transformers

- **Self-attention** lets every position directly access every other position, in parallel — overcoming the serial bottleneck and limited reach of RNNs.
- Each word is projected into a **query**, **key**, and **value**. Attention scores the query against all keys, softmaxes, and returns a weighted sum of values: $\text{softmax}(QK^\top/\sqrt{d_k})\,V$.
- **Causal masking** hides future tokens for language modelling. **Multi-head attention** runs several attentions in parallel to capture different relationships.
- A **transformer block** = multi-head attention + feedforward, each wrapped in a residual connection and layer norm. Blocks stack to form deep models.
- Inputs combine **token** and **position** embeddings. The **language modelling head** maps the final hidden state to a distribution over the vocabulary.
- Many NLP tasks become **next-word prediction**. Generation uses sampling (top-k, top-p, temperature). Training is **self-supervised**; performance follows **scaling laws**.
- LLMs carry real risks: **hallucination, bias, toxicity, misinformation, privacy, and copyright**.

Questions?

Contact Information	
Author	Emmanuel Djegou, PhD
Topic Area	Data Science, Statistical Modeling & Survival Analysis
Email	emmanueldjegou5@gmail.com
LinkedIn	linkedin.com/in/emmanuel-djegou-phd-5652b2254